

# **Gráfo de Escena e Implementación**

side quest para revisar la implementación de la cámara

# Cámara

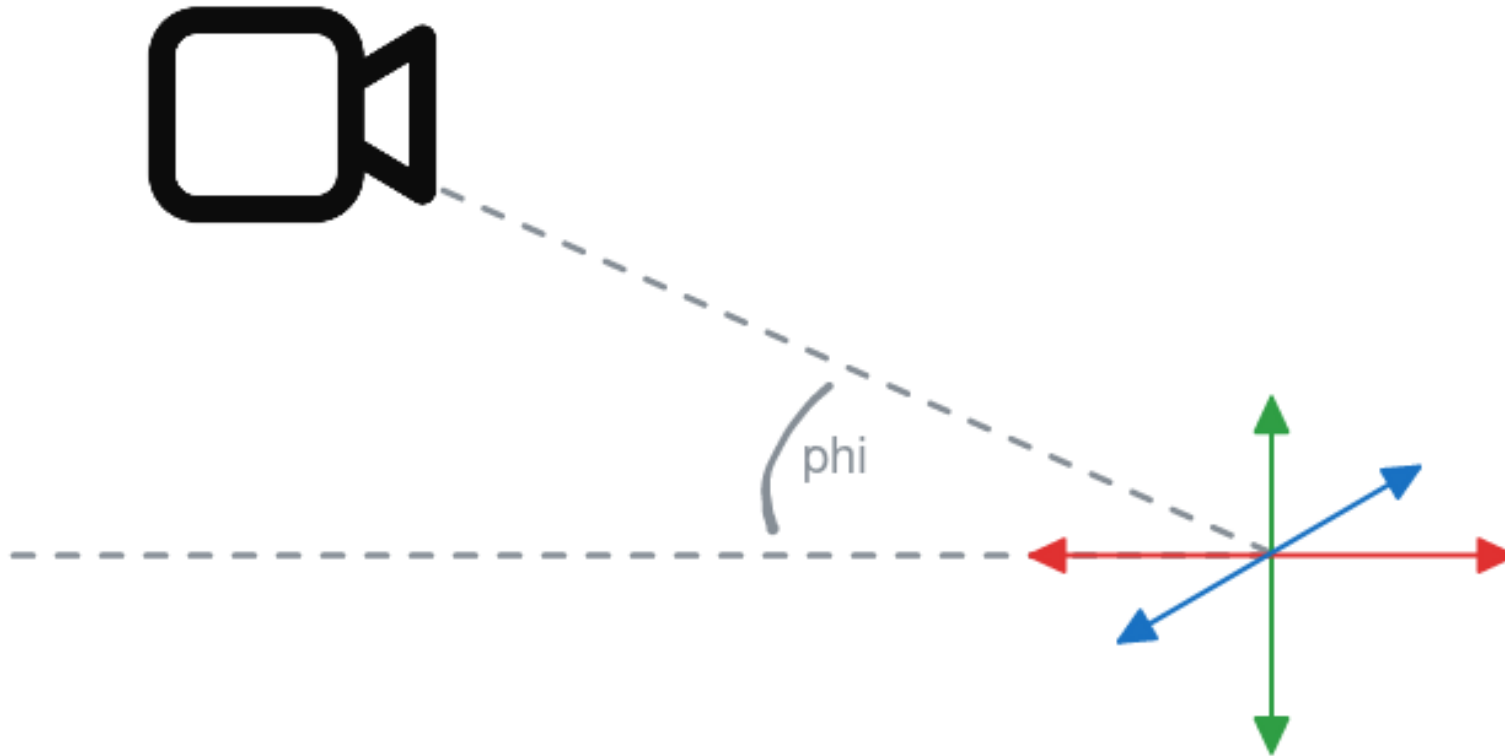
Vamos a incluir en nuestro modulo de helpers.py dos cosillas

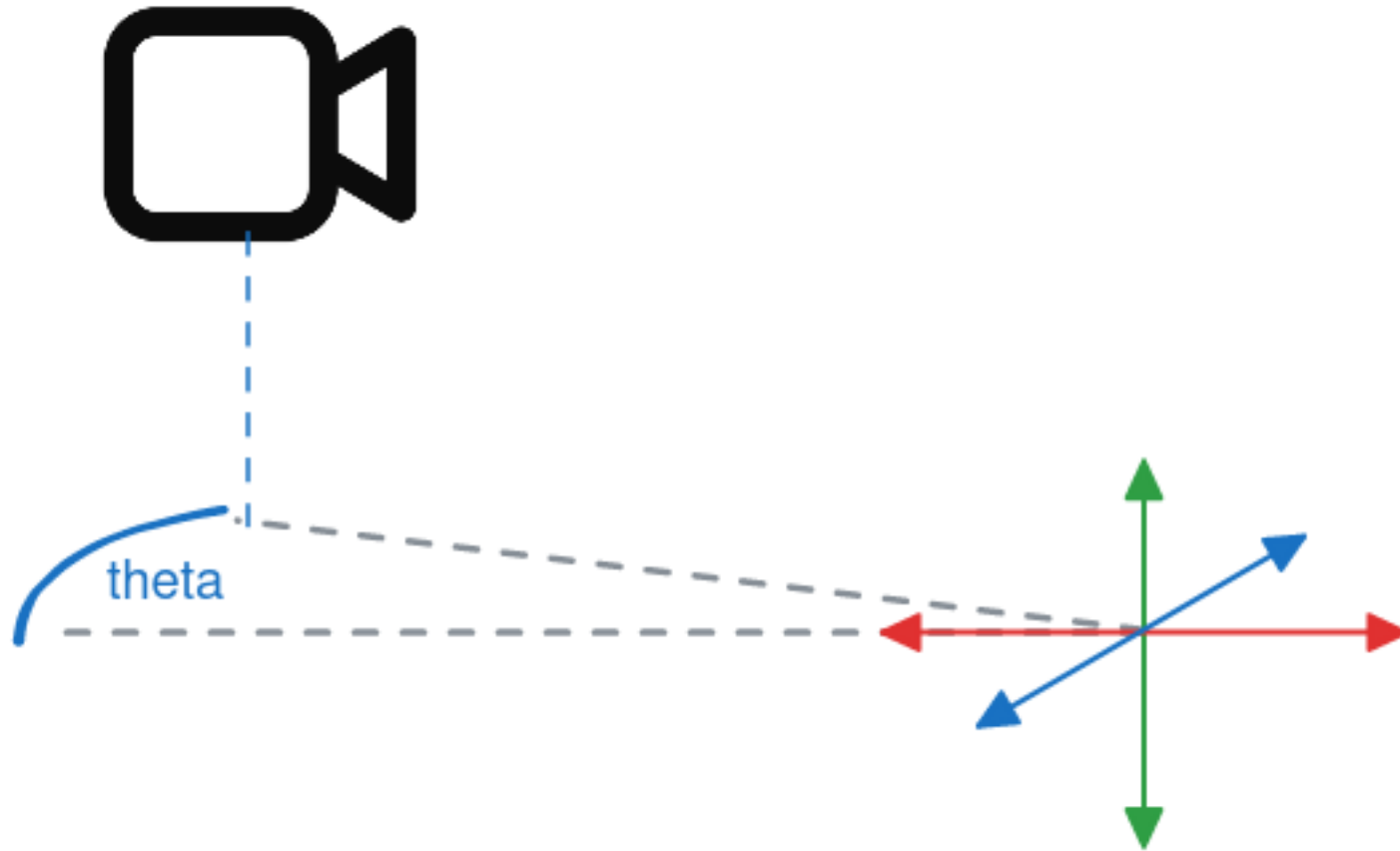
- `Camera(width, height, type)`
- `OrbitCamera(distancia, width, height, type)`

Podemos crear una cámara así:

```
camara = OrbitCamera(5, width, height, "perspective")  
camera.phi = np.pi / 4  
camera.theta = np.pi / 4
```

Estos valores  $\phi$  y  $\theta$  corresponden a los ángulos de la cámara.

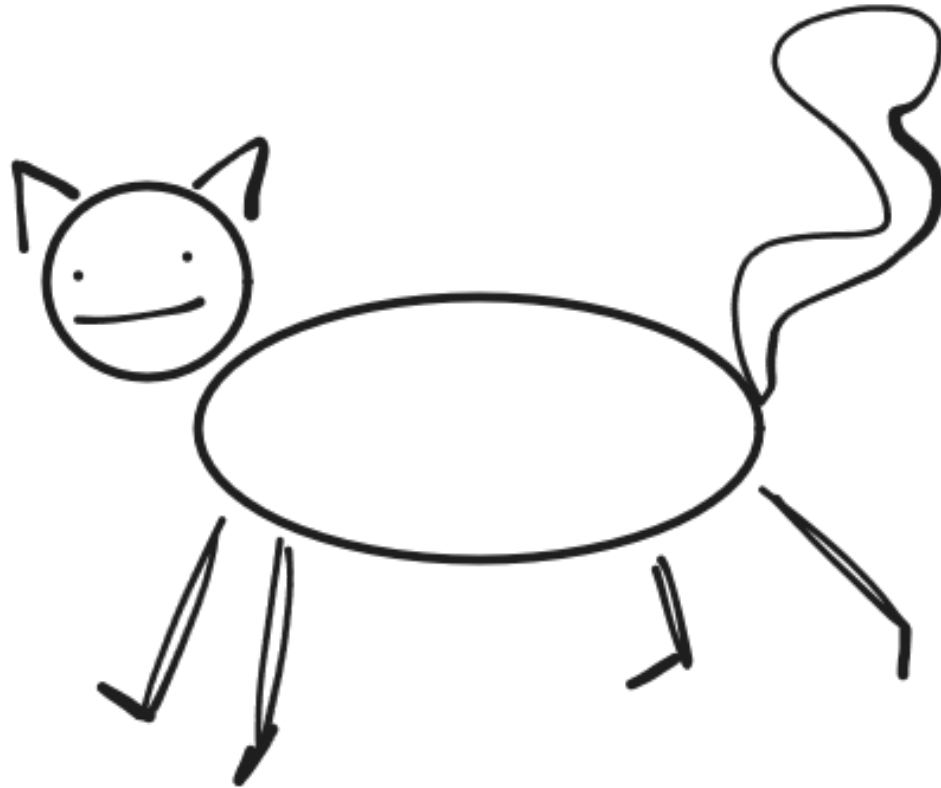




Ahora sí, gráfico de escena

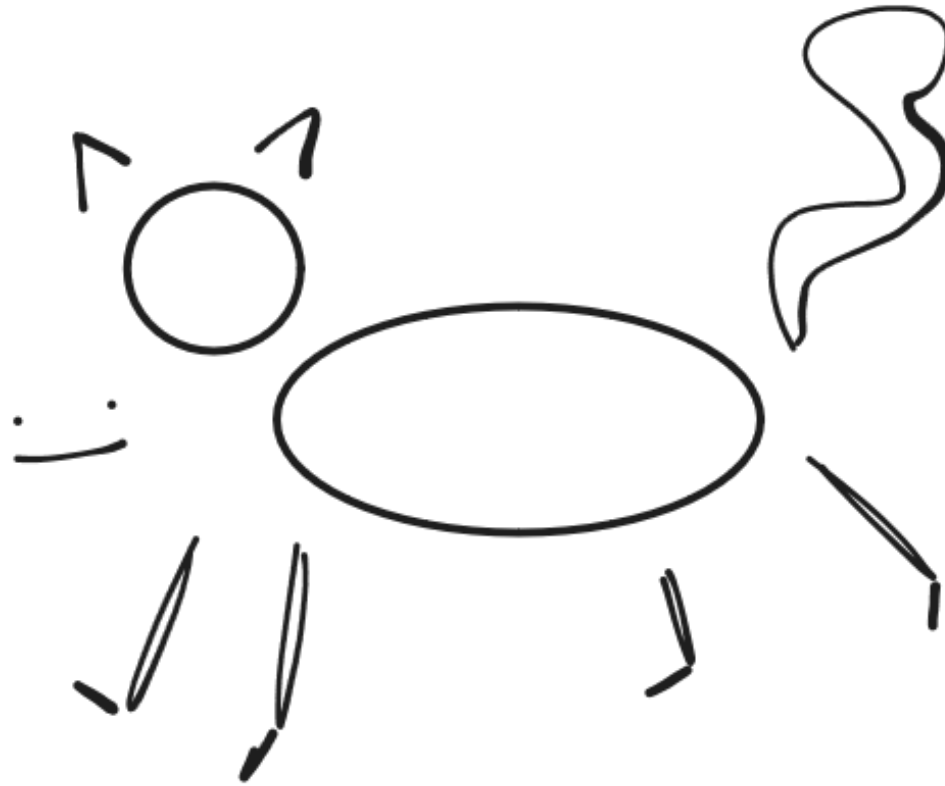
# **Motivación**

Digamos que queremos construir este gato

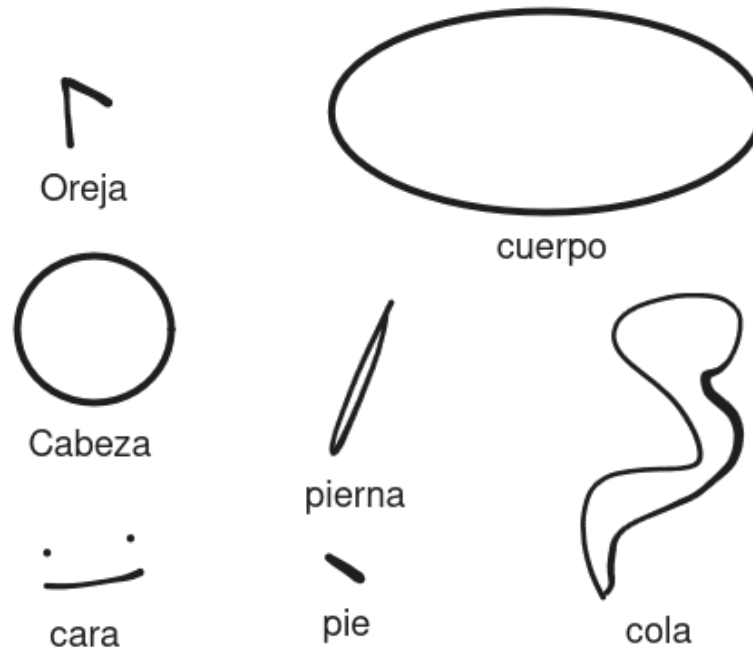




Pero queremos hacerlo parte por parte



Pie, pierna y oreja se repiten en distintas posiciones. Por lo que tenemos las siguientes partes únicas:



El gato es nuestro todo, nuestro conjunto completo. Pero podemos definirlo en base a subconjuntos de sus partes.

El gato es nuestro todo, nuestro conjunto completo. Pero podemos definirlo en base a subconjuntos de sus partes.

Por ejemplo, podemos definir **pata** como una **pierna** y un **pie** en cierta posición.

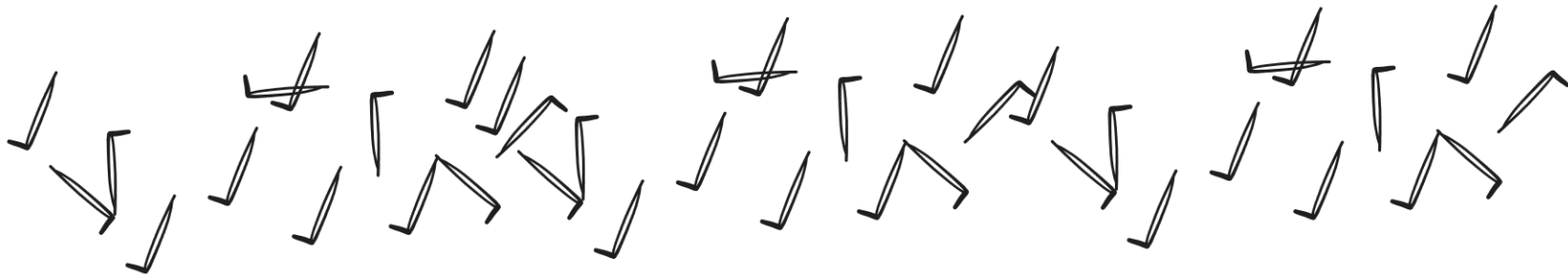


El gato es nuestro todo, nuestro conjunto completo. Pero podemos definirlo en base a subconjuntos de sus partes.

Por ejemplo, podemos definir **pata** como una **pierna** y un **pie** en cierta posición.



Y luego crear muchas patas.



Volviendo al gato, tendremos los siguientes subconjuntos:

- Pata = Pierna + pie
- Cuerpo = Torso + Cola + 4 Patas
- Orejas = Oreja + Oreja
- Cabeza = Cara + expresión + Orejas

Finalmente nuestro gato será Cabeza + Cuerpo

Yo definí estos conjuntos por un propósito educativo (se me dió la gana), podrían haber más formas de definir al gato...

Volviendo al gato, tendremos los siguientes subconjuntos:

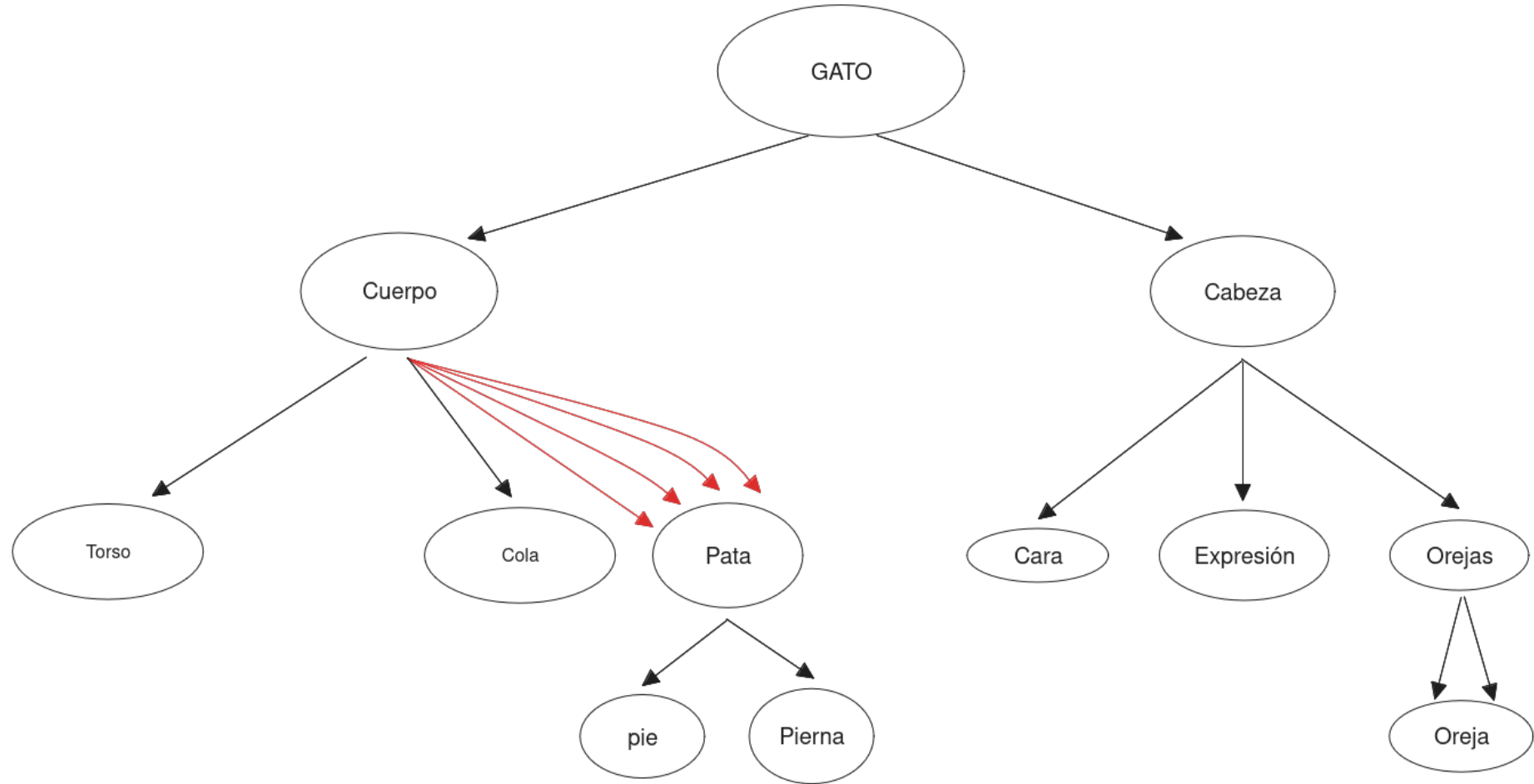
- Pata = Pierna + pie
- Cuerpo = Torso + Cola + 4 Patas
- Orejas = Oreja + Oreja
- Cabeza = Cara + expresión + Orejas

Finalmente nuestro gato será Cabeza + Cuerpo

Entonces, si decimos que **gato** es nuestra raíz... Y le siguen **Cabeza** y **Cuerpo**, y de cada uno salen mas objetos... es como un grafo...

# El Gráfo





## Gráfo de Escena

- La raíz es nuestra escena completa.
- Las hojas (nodos finales) representarán nuestros objetos bases.
- Los nodos internos (nodos que no son hojas ni raíz) serán los subconjuntos

No hay un consenso absoluto sobre cómo hacer un grafo de escena, podemos encontrar grafos donde los nodos internos son transformaciones... u otras cosas

El punto es reconocer que existe una **relación jerárquica** entre objetos.

## Implementación

Se incluye en el módulo `helpers.py` la clase `SceneGraph(camara)`

Y se usa así

```
graph = SceneGraph(camara)
```

## Añadir nodos

```
graph.add_node("nombre",  
               attach_to = ,  
               mesh =  
               color =  
               transformations =  
               scale =  
               position =  
               rotation =  
               )
```

## Actualizar cosas en el gráfo

`graph[objeto][parametro] = algo`

Por ejemplo

```
graph["cabeza"]["transform"] = tr.translate(1, 1, 1) @  
tr.scale(0.5, 0.5, 0.5)
```

# Dibujar todo lo del grafo

```
@controller.event  
def on_draw():  
    controller.clear()  
    graph.draw()
```

**Demo time**

intentionally left blank





# translate

/trɑːnzˈleɪt, trɑnzˈleɪt/

*verb*

1. express the sense of (words or text) in another language.  
"several of his books were **translated into** English"

Similar:

interpret

render

gloss

put

express

convert

change



2. move from one place or condition to another.  
"she had been translated from familiar surroundings to a foreign court"

Similar:

relocate

transfer

move

remove

shift

convey

transport

